

AI BUDGET MONITORING SYSTEM

A Project Report submitted for the Unified Mentor Internship

Prepared By:

Durgesh Bodke

Bachelor of Science (Computer Science)

CERTIFICATE

This is to certify that the project report entitled "AI Budget Monitoring System" is a bona fide work carried out by Mr. Durgesh Kishor Bodke as a part of the Unified Mentor Internship Program in partial fulfilment of the requirements of the internship.

The project has been successfully designed, developed, implemented, and documented under the guidance and supervision of Unified Mentor. The work presented in this report demonstrates the practical application of modern full-stack web development technologies, including Angular 22, Node.js, Express.js, and MongoDB Atlas, for developing an intelligent financial management platform.

The objective of this project was to design a secure and user-friendly AI-powered budget monitoring system capable of managing government departments, budget allocations, expenditure tracking, real-time alerts, analytical dashboards, report generation, and user management through an interactive web interface. The system integrates modern technologies to improve transparency, accountability, and decision-making in financial administration.

Project Guide / Mentor

Unified Mentor

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Unified Mentor for providing me with the opportunity to undertake this internship and develop the project titled "AI Budget Monitoring System." This internship has been an invaluable learning experience, enabling me to apply theoretical knowledge to real-world software development while enhancing my technical, analytical, and problem-solving skills.

I extend my heartfelt appreciation to my mentors and the technical support team at Unified Mentor for their constant guidance, encouragement, and constructive feedback throughout the project. Their valuable suggestions and professional insights helped me understand industry-standard development practices and motivated me to successfully complete this project.

I am also thankful to the faculty members of my Computer Science Department for providing a strong academic foundation and continuous encouragement during the course of this internship. Their support played a significant role in strengthening my understanding of full-stack web development and modern software engineering concepts.

Finally, I would like to thank everyone who directly or indirectly contributed to the successful completion of this project. This internship has significantly enhanced my practical knowledge of Angular 22, Node.js, Express.js, MongoDB Atlas, RESTful APIs, JWT Authentication, and full-stack development, and has strengthened my confidence in developing scalable and industry-ready software solutions.

ABSTRACT

The AI Budget Monitoring System is a comprehensive, full-stack web application engineered to revolutionize how organizations monitor budget allocation, track expenditures, and analyze financial utilization. Built on a modern MERN-like stack utilizing Angular 22 for the frontend, Node.js/Express for the backend API, and MongoDB Atlas for database management, the system ensures enterprise-grade security and scalability.

This project introduces a digital-first approach to budget monitoring, integrating rule-based artificial intelligence to provide predictive insights and automated anomaly detection. Key features include JWT-based user authentication, role-based access control, dynamic department management, real-time expense tracking, and intelligent alerting for spending spikes. Interactive visualization is achieved via Chart.js, offering clear financial summaries. This report thoroughly documents the system requirements, UML/DFD architectural designs, MongoDB database schemas, and implementation details corresponding to the 3-tier architecture.

TABLE OF CONTENTS

1 Introduction

1.1 Background

1.2 Existing System

1.3 Proposed System

1.4 Objectives

1.5 Scope

2 Organization Profile (Unified Mentor)

3 Literature Review

4 Requirement Analysis

5 System Design

5.1 System Architecture

5.2 Use Case Diagram

5.3 Activity Diagram

5.4 Data Flow Diagrams

6 Database Design

6.1 Entity Relationship Diagram

6.2 MongoDB Collections

7 Implementation

7.1 Folder Structure

7.2 REST API Architecture

7.3 JWT Authentication Flow

7.4 Budget Calculation & AI Logic

7.5 Deployment Architecture

8 Testing

9 Results & System UI

10 Conclusion

11 Future Scope

12 References

1. Introduction

1.1 Background

In the contemporary financial landscape, large organizations and government bodies manage immense sums of capital distributed across numerous departments. Effective financial governance requires complete transparency and immediate visibility into how funds are utilized. The AI Budget Monitoring System was conceived to address the inefficiencies present in standard financial workflows by providing a smart, centralized dashboard.

1.2 Existing System

Traditional budget management heavily relies on manual data entry into disparate spreadsheet software (e.g., MS Excel) or legacy on-premise ERP systems. These systems present several critical flaws:

- Delayed visibility leading to after-the-fact reporting.
- High susceptibility to human error during data entry.
- Lack of proactive predictive capabilities; they cannot forecast budget exhaustion before it happens.

1.3 Proposed System

The proposed AI Budget Monitoring System resolves these issues through an automated, intelligent web platform. Built with Angular 22, it provides a highly reactive Single Page Application (SPA) experience. As expenses are logged, they instantly update backend tallies in MongoDB. The system employs background logic to continuously evaluate expenditure velocity against time, generating preemptive alerts for potential over-utilization.

1.4 Objectives

- To establish a secure, centralized ledger for departmental budgets.
- To implement real-time expense tracking with an approval workflow.
- To integrate an AI insight engine to calculate financial health and alert on spending anomalies.

1.5 Scope

The current scope covers the end-to-end development of the platform, including secure JWT authentication, multi-role authorization (Admin vs. Employee), comprehensive CRUD modules for departments and budgets, file upload handling for expense receipts (via Multer), and graphical reporting dashboards.

2. Organization Profile

Unified Mentor is an educational and professional development platform that focuses on enhancing students' technical skills by providing industry-oriented internship programs and hands-on project experience. The organization aims to bridge the gap between academic learning and practical implementation by allowing interns to work on real-world software development projects using modern technologies and industry best practices.

The internship programs are designed to help participants gain practical knowledge of the complete software development lifecycle, including requirement analysis, system design, coding, testing, debugging, documentation, and deployment. Interns are encouraged to develop scalable and secure applications while following professional coding standards and modern development methodologies.

The AI Budget Monitoring System was developed as part of the Unified Mentor Internship Program using Angular 22, Node.js, Express.js, and MongoDB Atlas. The project demonstrates the implementation of a full-stack web application capable of managing departmental budgets, tracking expenses, generating reports, and providing AI-based financial insights through an interactive dashboard.

3. Literature Review

The rapid advancement of web technologies has significantly transformed the development of financial management applications. Earlier systems primarily relied on server-side rendering and desktop-based software, which often resulted in slower performance, limited scalability, and reduced user interaction. These traditional systems also required considerable manual effort for budget tracking, expense management, and report generation, making them less suitable for organizations handling large volumes of financial data.

Modern web frameworks such as Angular 22 have revolutionized frontend development by enabling developers to build highly responsive, component-based, and dynamic user interfaces. These capabilities make Angular an excellent choice for developing real-time financial dashboards and interactive reporting systems.

On the backend, technologies like Node.js and Express.js have replaced many traditional monolithic architectures with lightweight, RESTful API-based systems. This approach allows different application modules to communicate efficiently, improves scalability, and simplifies maintenance. REST APIs also enable easy integration with other services and support future expansion of the application.

For database management, MongoDB Atlas provides a flexible NoSQL database solution that efficiently stores and manages structured as well as semi-structured financial data. Its document-oriented architecture supports rapid data retrieval, high performance, and easy scalability, making it well suited for applications where data requirements may evolve over time.

The AI Budget Monitoring System developed during this internship follows these modern software development practices by combining Angular 22, Node.js, Express.js, MongoDB Atlas, JWT Authentication, and Chart.js to deliver a secure, scalable, and user-friendly financial management platform. The project demonstrates how contemporary web technologies can be effectively integrated to provide real-time budget monitoring, analytical reporting, and intelligent financial insights for government departments and organizations.

4. Requirement Analysis

Software Requirement Analysis encompasses identifying the necessary dependencies, security standards, and operational features to make the system viable.

Functional Requirements:

- Secure login and registration using encrypted passwords (bcrypt) and JWT.
- Role-Based Access Control restricting budget allocation to Administrators.
- Automated validation preventing expense submission if the amount exceeds the remaining budget.

Non-Functional Requirements:

- Responsiveness: The Angular 22 UI must function seamlessly on desktops and tablets.
- Security: API endpoints must be protected against unauthorized access and SQL/NoSQL injection.
- Performance: Dashboard charts must render within 2 seconds of querying the database.

5. System Design

5.1 System Architecture

The system utilizes a 3-Tier Architecture separating the Presentation Layer (Angular 22), Application Layer (Node.js/Express API), and Data Layer (MongoDB).

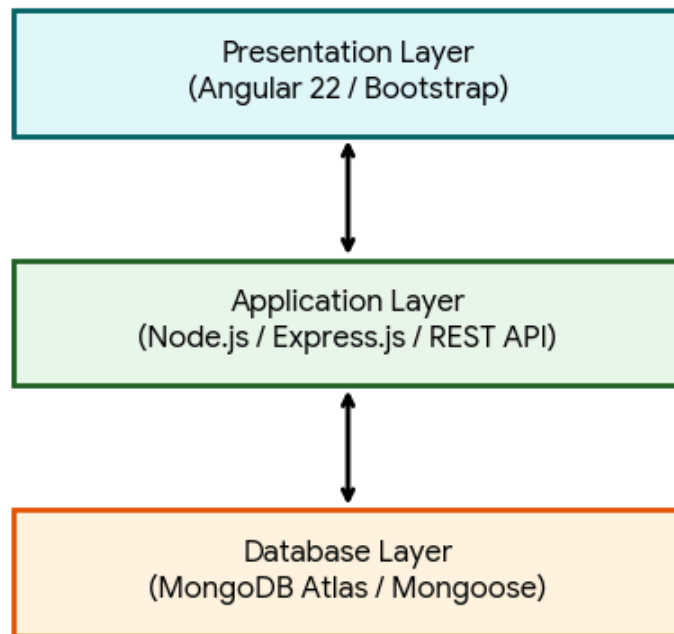


Figure 1: 3-Tier System Architecture Diagram

5.2 Use Case Diagram

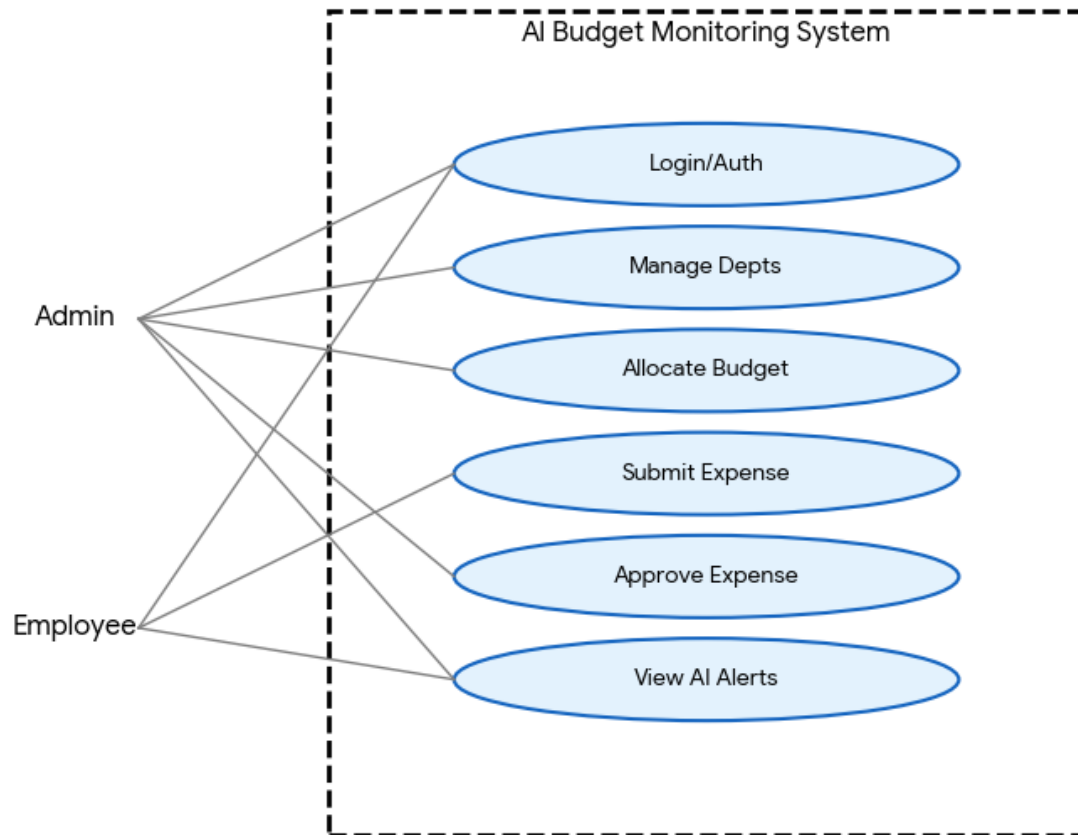


Figure 2: UML Use Case Diagram

5.3 Activity Diagram

Activity Diagram: Expense Approval

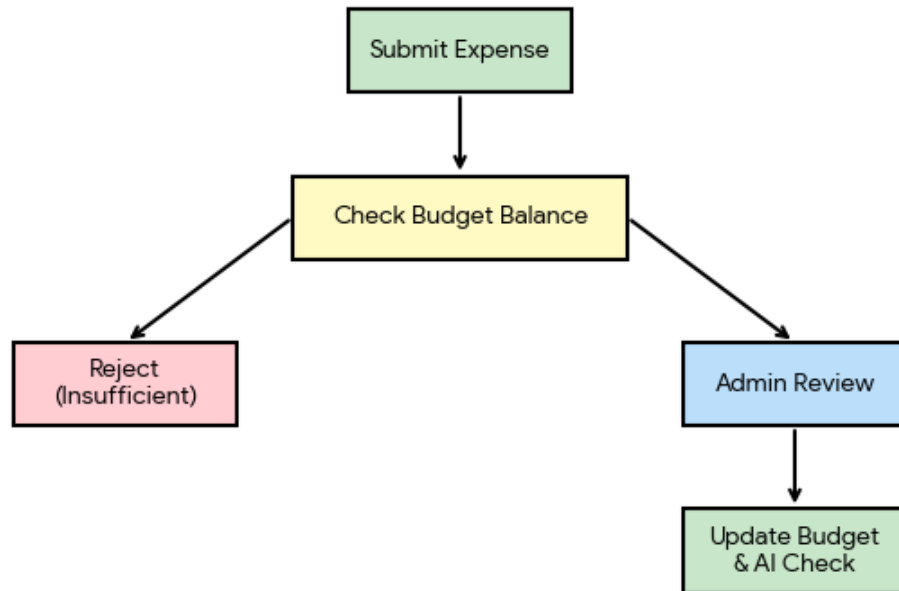


Figure 3: Activity Diagram for Expense Approval

5.4 Data Flow Diagrams

DFD Level 0 (Context Diagram)

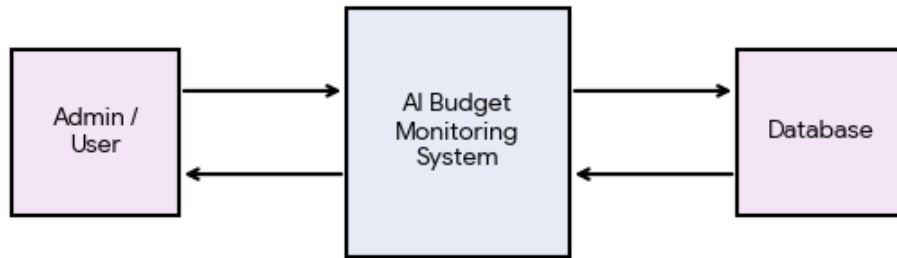


Figure 4: DFD Level 0 (Context Diagram)

6. Database Design

6.1 Entity Relationship Diagram

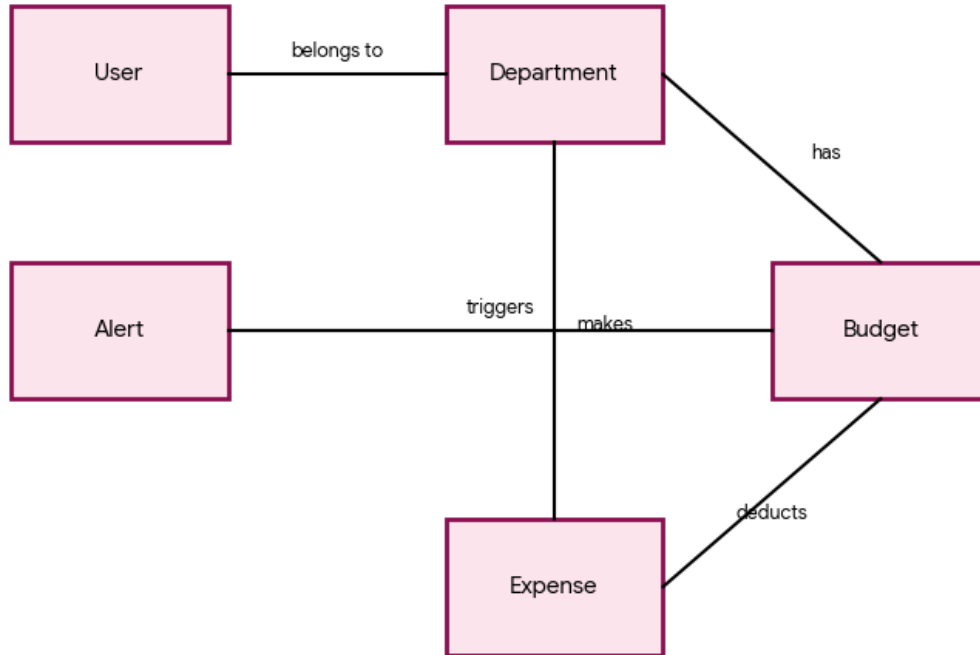


Figure 5: Entity Relationship (ER) Diagram

6.2 MongoDB Collections

The NoSQL database relies on deeply integrated collections via Mongoose ObjectIds.

- Users: Stores userId, fullName, email, bcrypt-hashed password, and role (Admin/Employee).
- Departments: Stores deptId, name, code, headOfDept, and operational status.
- Budgets: Stores references to Departments, financialYear, totalAllocated, totalUtilized, and dynamically calculated remaining funds.
- Expenses: Stores references to Budgets, amount, category, paymentMethod, status (Pending/Approved), and receipt file paths.
- Alerts: Stores AI-generated records detailing severity (Critical/High/Medium) and insight messages.

7. Implementation

7.1 Folder Structure

The project is cleanly separated into client and server workspaces.

client/ (Angular 22)

- |— src/app/layouts/ (Dashboard, Modals)
- |— src/app/services/ (HTTP API connectors)
- |— src/app/guards/ (Auth routing protection)

server/ (Node.js)

- |— controllers/ (Business logic)
- |— models/ (Mongoose schemas)
- |— routes/ (Express router)
- |— middleware/ (JWT validation, Multer uploads)

7.2 REST API Architecture

The backend exposes standard RESTful endpoints returning JSON payloads. All sensitive routes (e.g., POST /api/budgets/allocate) are wrapped in custom authentication middleware that verifies the provided JWT Bearer token and checks the user role.

7.3 JWT Authentication Flow

Upon successful login, the Node.js server signs a JSON Web Token using a secret key and a payload containing the user ID and role. This token is sent to the Angular 22 client, which stores it in local storage. HTTP Interceptors in Angular automatically attach this token to the headers of all subsequent outbound requests.

7.4 Budget Calculation & AI Insights Logic

The core budget workflow guarantees mathematical consistency. When an admin approves a pending expense, a MongoDB transaction ensures that the budget's `totalUtilized` increases and `remaining` decreases simultaneously.

The AI Logic is triggered immediately post-update. It evaluates the utilization percentage. If $(\text{totalUtilized} / \text{totalAllocated}) * 100 > 90$, the system automatically inserts a "Critical" alert document into the database, instantly warning administrators of impending budget exhaustion.

7.5 Deployment Architecture

In a production environment, the Angular 22 application can be hosted on static CDNs (like Vercel or Firebase Hosting), while the Node.js server operates on PaaS providers like Heroku or Render. Database operations are handled seamlessly by MongoDB Atlas in the cloud.

8. Testing

Systematic testing was executed to validate data integrity and security.

1. Unit Testing: Individual Angular components were verified for rendering accuracy and form validation (e.g., ensuring expense amounts cannot be negative).
2. API Integration Testing: Postman was used extensively to hit endpoint routes. Security checks confirmed that requests lacking a valid JWT received a 401 Unauthorized response.
3. Scenario Testing: Simulating an over-budget transaction. When an employee attempted to submit a 50,000 INR expense against a budget containing only 10,000 INR remaining, the backend accurately blocked the transaction and threw a 400 Bad Request error, which was gracefully handled by the Angular UI alerting the user.

9. Results

The successful completion of the development and implementation phases resulted in a secure, responsive, and feature-rich web application for budget monitoring and financial management. The AI Budget Monitoring System effectively integrates modern full-stack technologies, including Angular 22, Node.js, Express.js, and MongoDB Atlas, to provide a centralized platform for managing departmental budgets, expenses, reports, and user access.

The application was thoroughly tested to ensure the accuracy of budget calculations, secure user authentication, seamless communication between the frontend and backend, and efficient data management. All major modules were successfully implemented and function as intended, providing users with an intuitive interface and real-time access to financial information.

The following section presents the implemented modules of the system along with screenshots of the user interface. Each screenshot highlights the practical implementation of Angular 22 components, responsive design principles, RESTful API integration, interactive data visualization using Chart.js, and secure backend communication. The screenshots also demonstrate the overall functionality, usability, and effectiveness of the developed application in managing organizational budgets and supporting informed financial decision-making.

The successful execution of all modules confirms that the proposed system meets the project objectives by providing an efficient, scalable, and user-friendly solution for budget monitoring and expenditure management.

9.1 System Login Interface

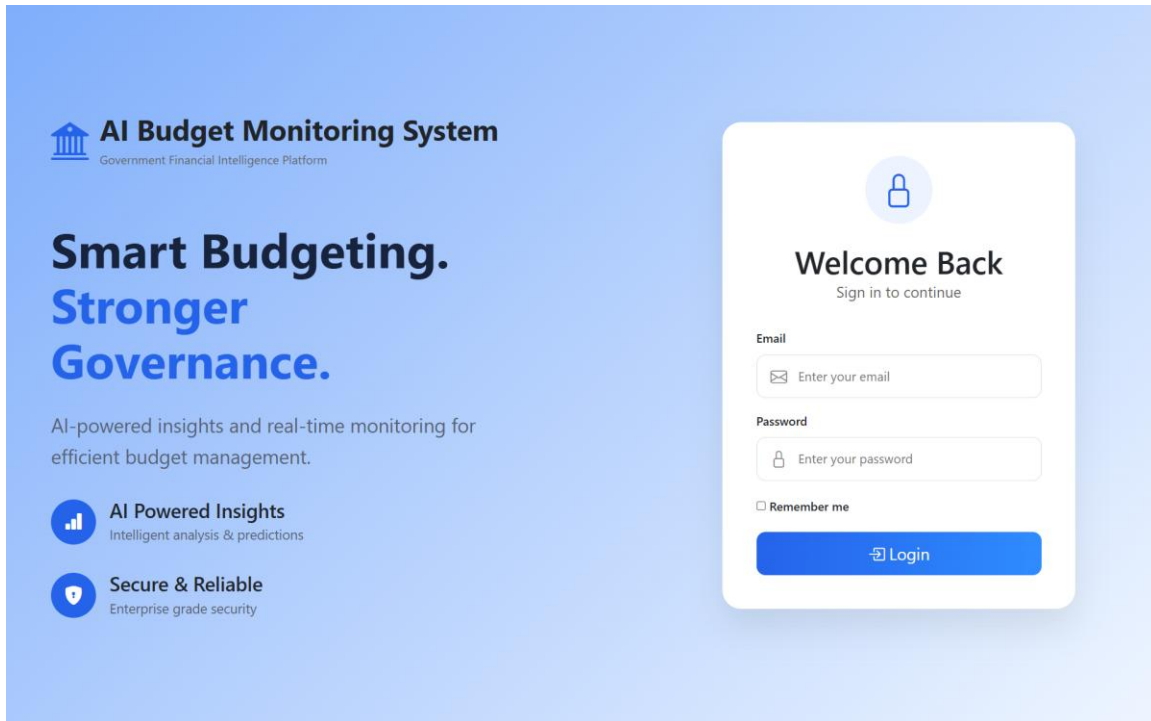


Figure 6: System Login Interface

The authentication gateway. It requires valid email and password credentials. Passwords are encrypted using bcrypt on the server. Upon authentication, a JWT is issued to the client.

9.2 Main Dashboard

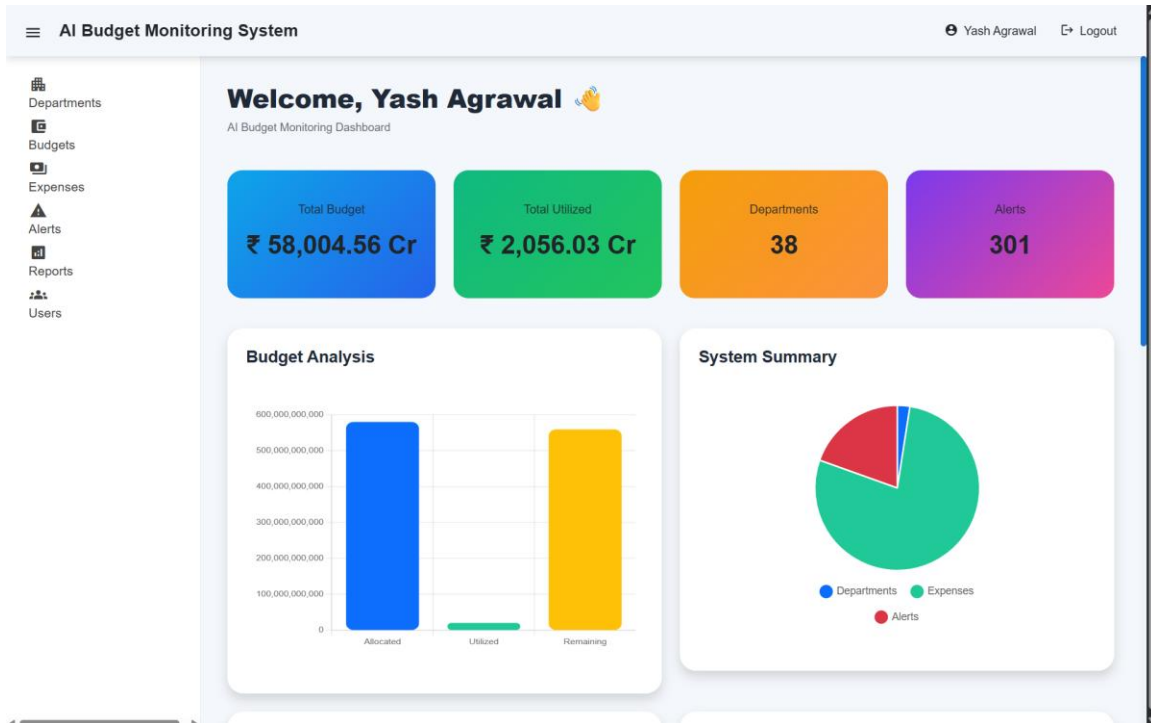


Figure 7: Main Dashboard

The primary view upon login. It highlights major KPIs: Total Budget, Total Utilized, active Departments, and Alert counts. It integrates Chart.js to render real-time side-by-side bar charts (Allocated vs. Utilized) and pie charts for system summary distribution.

9.3 Recent Activity Dashboard

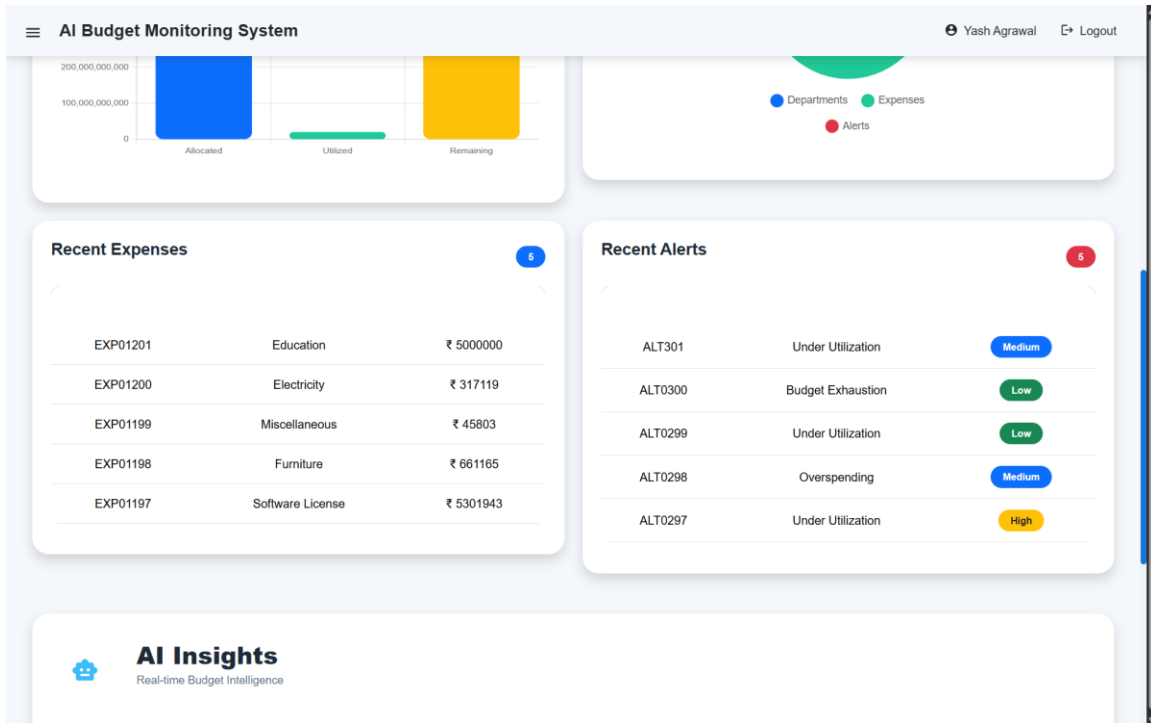


Figure 8: Recent Activity Dashboard

A sub-section of the main dashboard displaying continuous feeds of Recent Expenses and Recent Alerts. This ensures administrators immediately see the latest financial activities and potential warnings without navigating away from the home screen.

9.4 AI Insights Dashboard

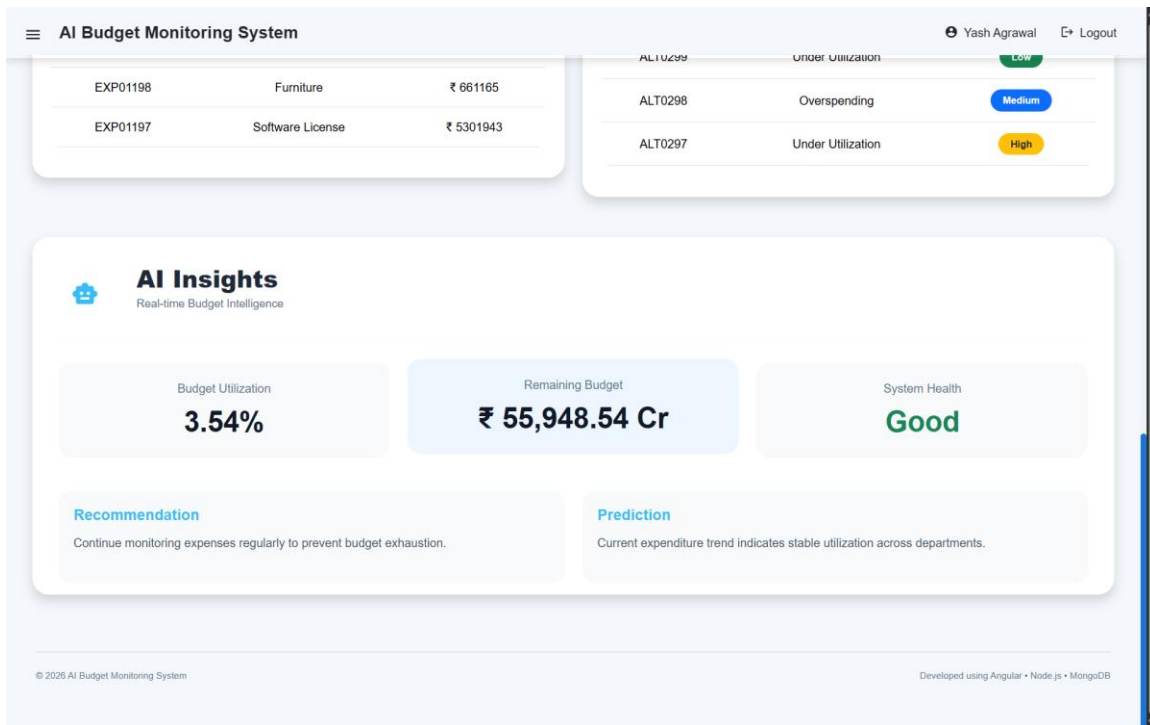


Figure 9: AI Insights Dashboard

The intelligent component of the system. It calculates absolute budget utilization percentages and provides automated recommendations (e.g., "Continue monitoring expenses regularly") and predictive analysis regarding expenditure trends across departments.

9.5 Department Management

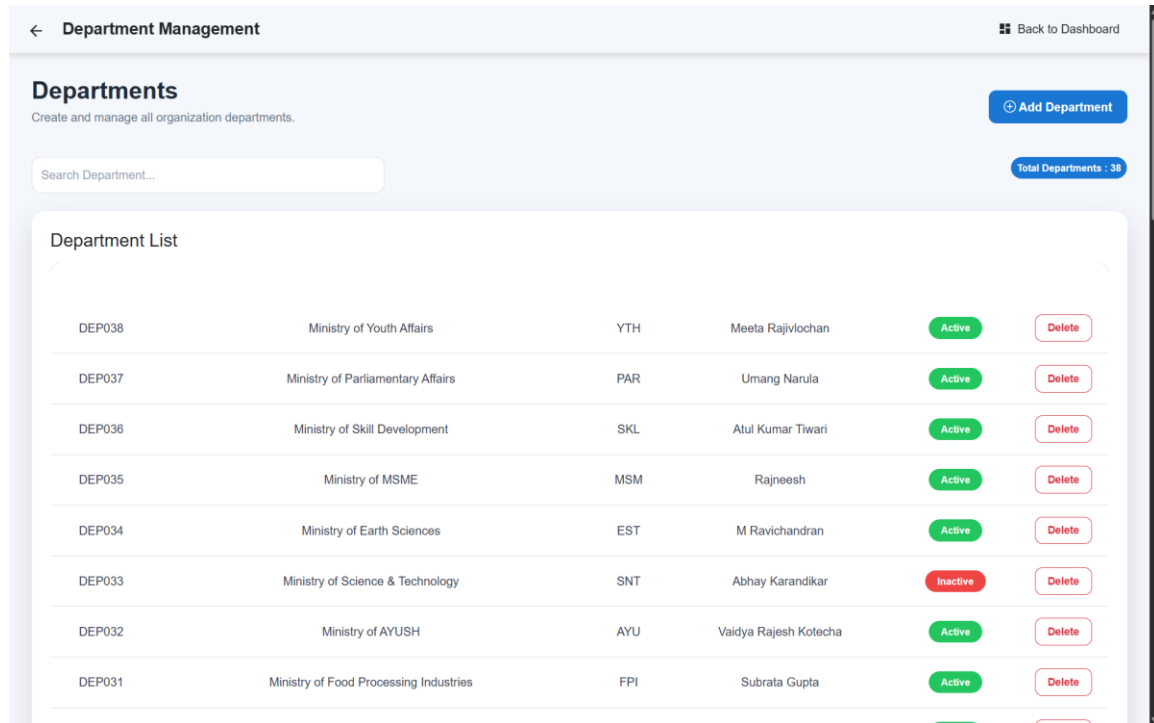


Figure 10: Department Management

The CRUD interface for handling organizational structures. Administrators can add new departments, view current heads of departments, verify their active/inactive status, and securely delete outdated entries.

9.6 Budget Allocation

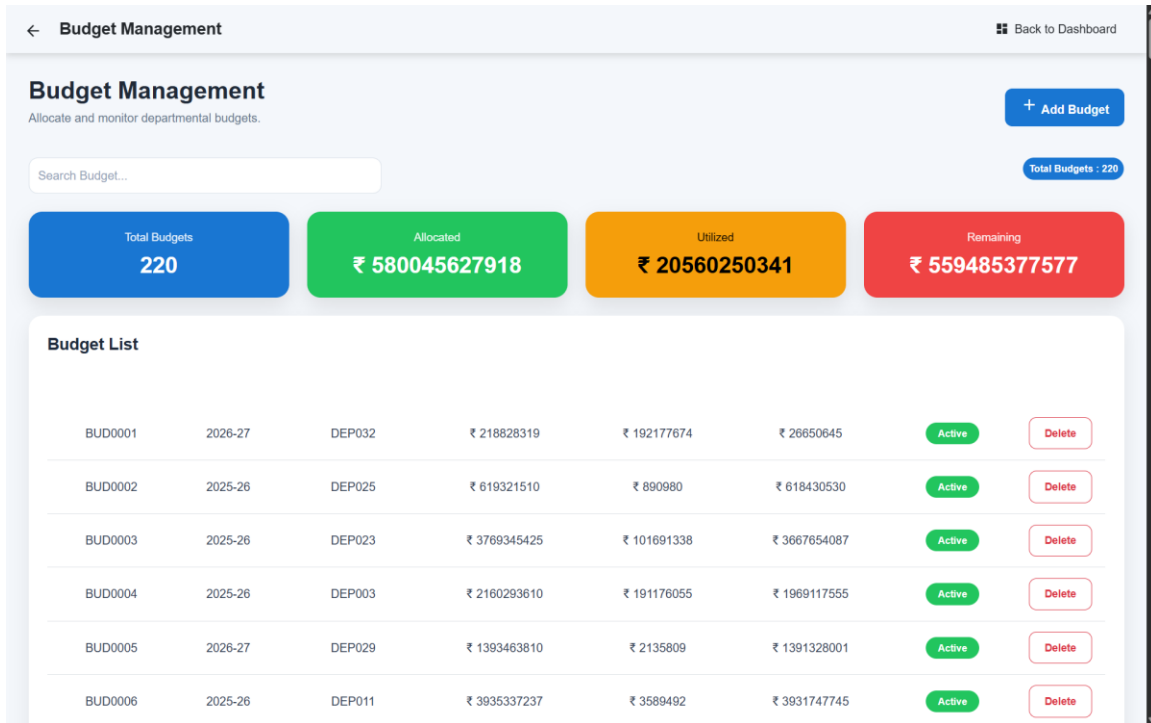


Figure 11: Budget Allocation

The financial ledger connecting funds to departments based on the financial year. It highlights total budgets allocated alongside green, yellow, and red KPI cards denoting Allocated, Utilized, and Remaining funds respectively.

9.7 Expense Tracking

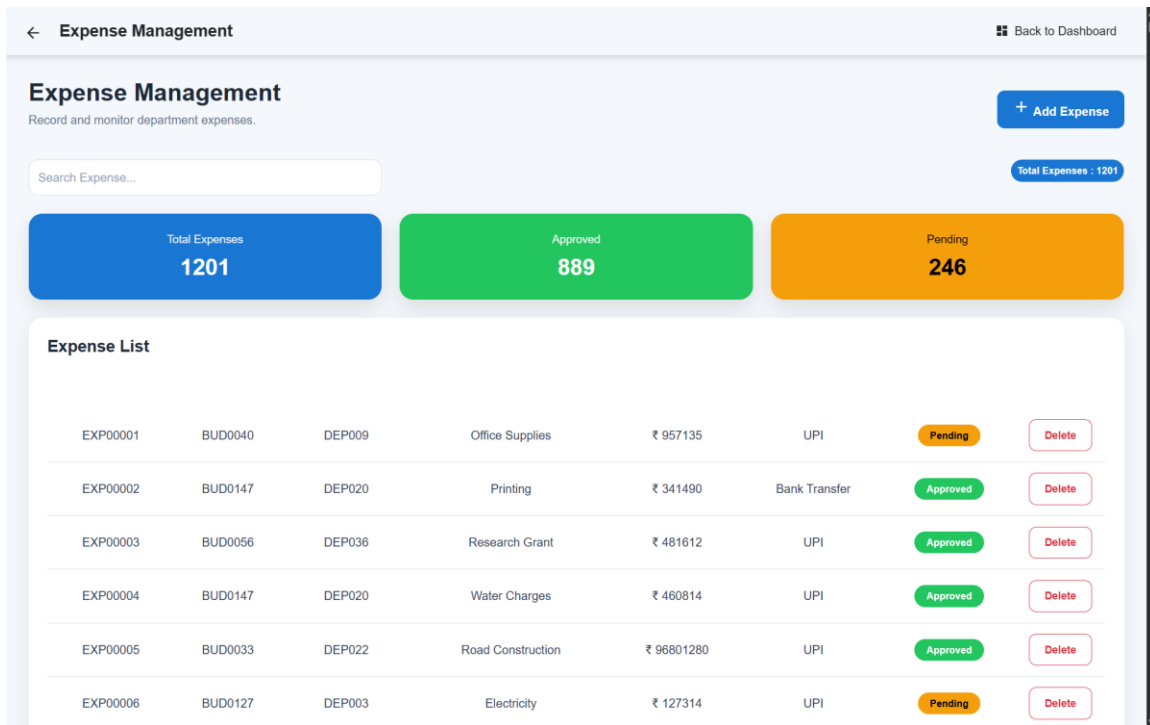


Figure 12: Expense Tracking

The operational core where transactions are tracked. Expenses display status pills (Pending/Approved) alongside categorization and payment methods. Forms validate against the remaining budget before allowing new submissions.

9.8 Alert Monitoring

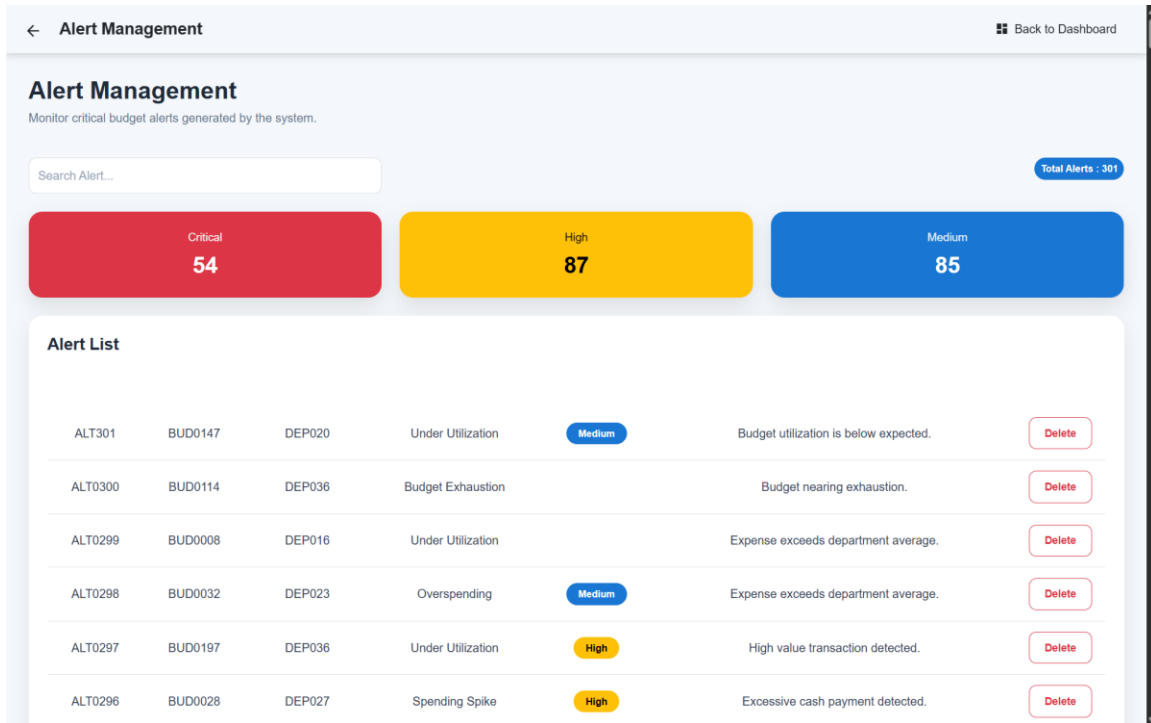


Figure 13: Alert Monitoring

The visual output of the backend AI heuristics. Alerts are classified by severity (Critical in red, High in yellow, Medium in blue). This module provides direct textual insights, such as "Budget nearing exhaustion" or "Excessive cash payment detected".

9.9 System Reports

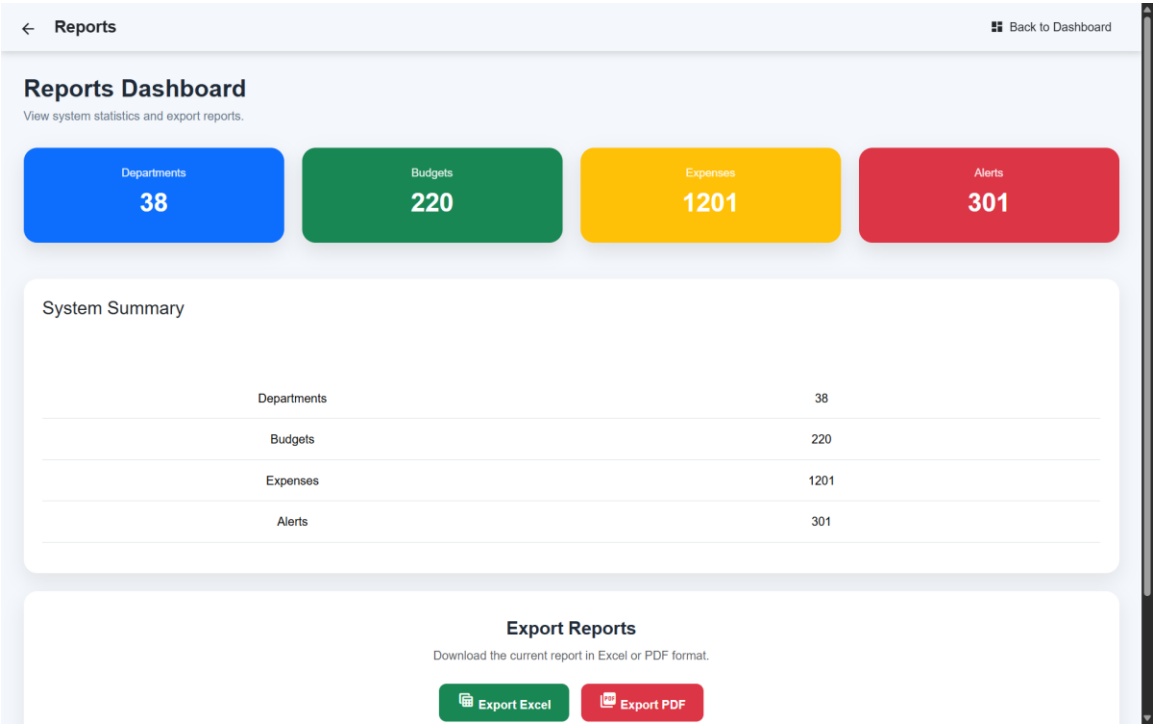


Figure 14: System Reports

The export hub. It provides an aggregate view of all data points and incorporates features to download the current system state into Excel or PDF formats for external audits and stakeholder meetings.

9.10 User Management

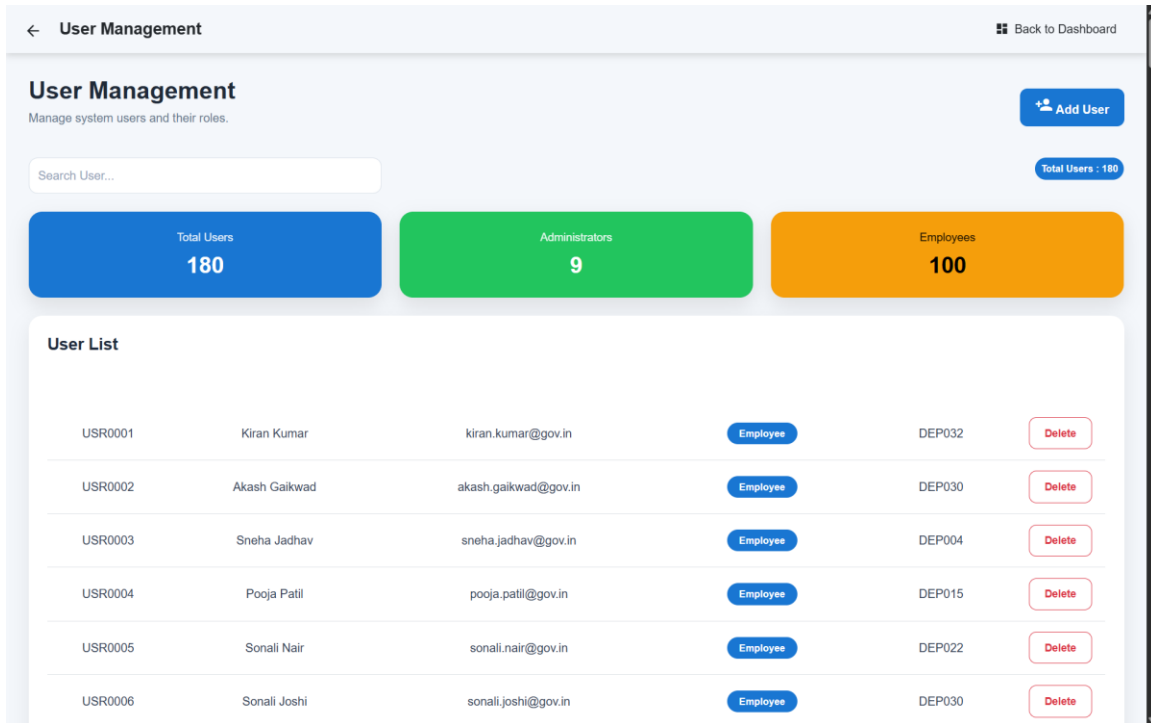


Figure 15: User Management

The Role-Based Access Control (RBAC) interface. Accessible only to top-level administrators, this screen manages employee and admin accounts, overseeing access to the platform and ensuring strict internal security.

10. Conclusion

The AI Budget Monitoring System successfully fulfills all initial project objectives, delivering a highly responsive, secure, and intelligent financial platform. By leveraging the power of Angular 22 for a seamless user experience and Node.js with MongoDB for robust data processing, the system completely modernizes traditional budget tracking.

The incorporation of automated rule-based AI significantly elevates the platform, transforming it from a passive ledger into a proactive advisory tool. The system effectively prevents budget exhaustion, enforces strict access control through JWT, and provides stunning, easily digestible data visualization, ultimately promoting stronger financial governance within any organization.

11. Future Scope

While the current system provides a comprehensive baseline, several enterprise-level enhancements can be incorporated in future iterations:

1. **Advanced Machine Learning Forecasting:** Transitioning from rule-based heuristics to actual predictive models (like ARIMA) to forecast next year's required budgets based on historical patterns.
2. **Automated OCR for Receipts:** Integrating Optical Character Recognition to automatically parse vendor names and amounts from uploaded image receipts.
3. **Multi-level Approval Workflows:** Implementing dynamic routing where transactions over 1,000,000 INR require secondary approval from a CFO.
4. **Banking API Integrations:** Connecting directly with banking portals to pull real-time transaction data and auto-reconcile ledgers.

12. References

- [1] Angular Team. "Angular 22 Documentation." Available: <https://angular.io/docs>

- [2] Node.js Foundation. "Node.js API Documentation." Available:
<https://nodejs.org/docs/>

- [3] MongoDB Inc. "MongoDB Atlas and Document Schema Design." Available:
<https://www.mongodb.com/docs/atlas/>

- [4] Express API Reference. "Express - Fast, unopinionated web framework for Node.js."
Available: <https://expressjs.com/>

- [5] Mongoose ODM. "Elegant MongoDB object modeling for Node.js." Available:
<https://mongoosejs.com/>

- [6] Auth0. "Introduction to JSON Web Tokens (JWT)." Available:
<https://jwt.io/introduction>

- [7] Chart.js Contributors. "Simple yet flexible JavaScript charting." Available:
<https://www.chartjs.org/docs/>

- [8] npm. "bcrypt - A library to help you hash passwords." Available:
<https://www.npmjs.com/package/bcrypt>

- [9] Bootstrap Team. "Bootstrap 5 Documentation." Available:
<https://getbootstrap.com/docs/>